Asymptotic Complexity

Comp Sci 214, Spring 2025

Don’t forget to check in on Youhere!

Announcements

- Homework 1 feedback out, resubmission due Thursday
  - Got something wrong? Don't panic; resubmit!
  - Small slip-up? No problem, easy to fix!
  - Larger mistakes? You still have things to learn!
    - And you should get credit for learning them!
  - **Regression testing**: don't fail same the tests again!

- Homework 1 self-evaluation out, also due Thursday
  - **Goal**: Evaluate your code beyond functional correctness
  - Done with your **first** submission

Announcements

- Homework 1 feedback out, resubmission due Thursday
  - Got something wrong? Don't panic; resubmit!
  - Small slip-up? No problem, easy to fix!
  - Larger mistakes? You still have things to learn!
    - And you should get credit for learning them!
  - **Regression testing**: don't fail same the tests again!

- Homework 1 self-evaluation out, also due Thursday
  - **Goal**: Evaluate your code beyond functional correctness
  - Done with your **first** submission

- Testing: wide variance! (to put it mildly)
  - See supplementary materials on Canvas
  - Coming homework 2: mutation testing
- Academic integrity

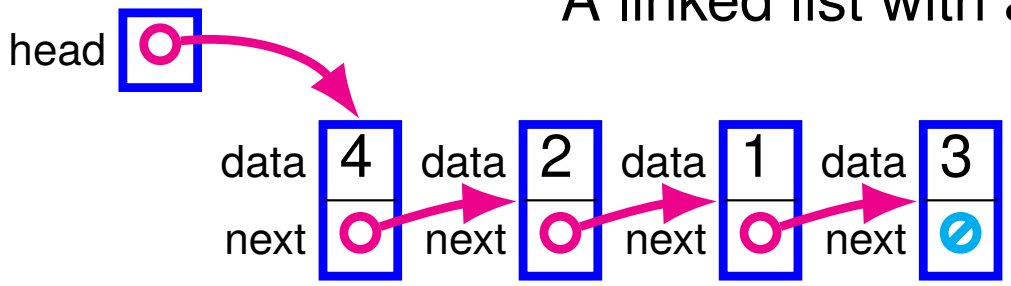
Announcements

Worksheets are out (two Canvas quizzes)

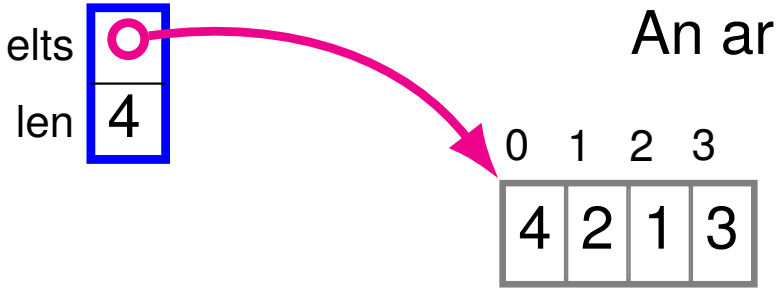
- One on asymptotic complexity (today)
- One on graphs (Chapter 10 of the textbook (on Canvas))
  - We will start discussing the topic on April 29th
  - We will assume you've read the chapter, pick up from there
- Instant feedback and 10 attempts
  - So you can understand your mistakes and learn from them!
  - **Pro tip**: trial and error is not learning. It's the opposite.

Comparison: Two Different Sequence DSs

A linked list with a head arrow only



An array that carries its length



Different representations $\rightarrow$ different costs for operations

Today let's make this precise

Comparison: Two Different Sequences

How long does it take to…

	linked list	array
Get the last elt of a 100-elt...	1,025µs	1.3µs
Add an elt to the front of a 100-elt...	9.7µs	3,070µs

Comparison: Two Different Sequences

How long does it take to…

	linked list	array
Get the last elt of a 100-elt...	1,025µs	1.3µs
Add an elt to the front of a 100-elt...	9.7µs	3,070µs

Ok, but what if I get a faster computer?

Or what if I have a 1000-element sequence? *n*-element?

Comparison: Two Different Sequences

How long does it take to…

	linked list	array
Get the last elt of a 100-elt...	1,025µs	1.3µs
Add an elt to the front of a 100-elt...	9.7µs	3,070µs

Ok, but what if I get a faster computer?

Or what if I have a 1000-element sequence? *n*-element?

Running an experiment for every question is not feasible.

We need a *predictive model* to answer Qs without measuring!

Our Model: Computational Complexity

- Look at all the individual actions the operation does
  - Some take a *fixed* amount of time (*independent* of input)
  - Some take a *variable* amount of time (*dependent* on input)
- Some actions are done *once*, some happen *many times*
  - In some cases, number of times *depends on input*!
- Sum everything up: that's our cost!
  - Will be a *function* of our input
  - Can be used to make *predictions* for different inputs!
  - In particular: predict how **time increases** as **inputs grow**!

```
def array_last(array):
    return array.elts[array.len - 1]

def list_last(lst):
    let cur = lst.head
    while cur.next is not None:
        cur = cur.next
    return cur.data
```

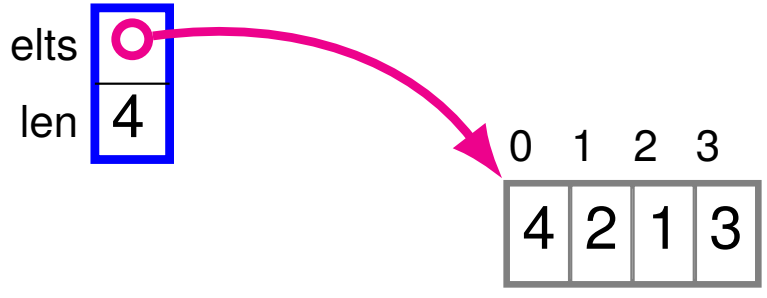
Please forgive the lack of error checking.

Modeling `array_last`

```
def array_last(array, n):
    return array.elts[array.len - 1]
```

$T_{thing}(n)$ = time to do *thing* on an input of size n

T_{thing} = time to do *thing* that takes a fixed amount of time



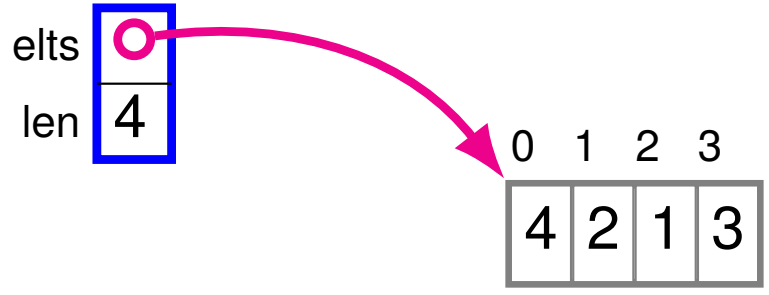
Modeling `array_last`

```
def array_last(array, n):
    return array.elts[array.len - 1]
```

$T_{thing}(n)$ = time to do *thing* on an input of size n

T_{thing} = time to do *thing* that takes a fixed amount of time

$$T_{\text{array\_last}}(n) =$$



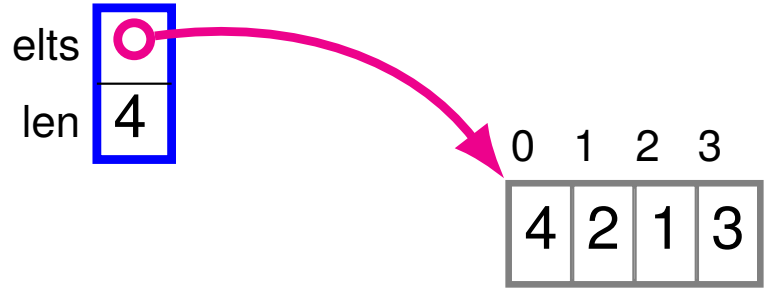
Modeling `array_last`

```
def array_last(array, n):
    return array.elts[array.len - 1]
```

$T_{thing}(n)$ = time to do *thing* on an input of size n

T_{thing} = time to do *thing* that takes a fixed amount of time

$$T_{\texttt{array\_last}}(n) = T_{\texttt{.elts}}$$



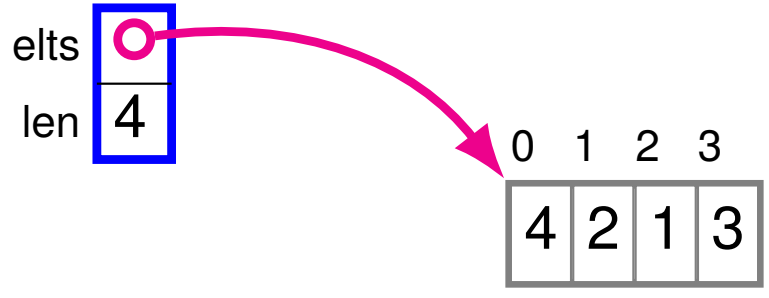
Modeling `array_last`

```
def array_last(array, n):
    return array.elts[array.len - 1]
```

$T_{thing}(n)$ = time to do *thing* on an input of size n

T_{thing} = time to do *thing* that takes a fixed amount of time

$$T_{\texttt{array\_last}}(n) = T_{\texttt{.elts}} + T_{\texttt{.len}}$$



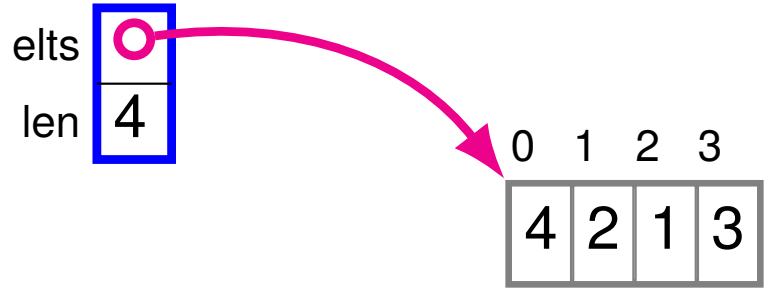
Modeling `array_last`

```
def array_last(array, n):
    return array.elts[array.len - 1]
```

$T_{thing}(n)$ = time to do *thing* on an input of size n

T_{thing} = time to do *thing* that takes a fixed amount of time

$$T_{\texttt{array\_last}}(n) = T_{\texttt{.elts}} + T_{\texttt{.len}} + T_{\text{-1}}$$



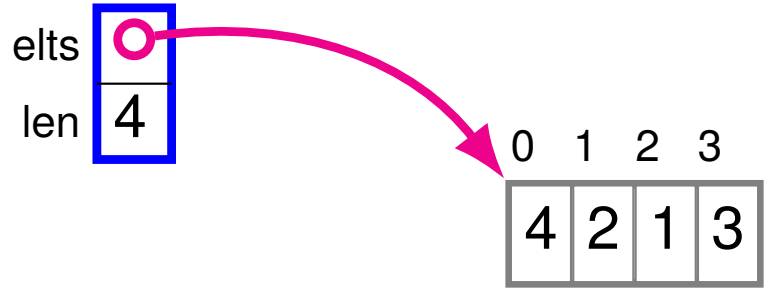
Modeling `array_last`

```
def array_last(array, n):
    return array.elts[array.len - 1]
```

$T_{thing}(n)$ = time to do *thing* on an input of size n

T_{thing} = time to do *thing* that takes a fixed amount of time

$$T_{\texttt{array\_last}}(n) = T_{\texttt{.elts}} + T_{\texttt{.len}} + T_{\texttt{-1}} + T_{\texttt{[]}}$$



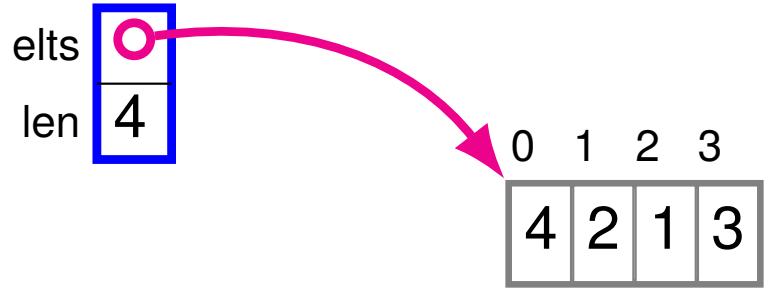
Modeling `array_last`

```
def array_last(array, n):
    return array.elts[array.len - 1]
```

$T_{thing}(n)$ = time to do *thing* on an input of size n

T_{thing} = time to do *thing* that takes a fixed amount of time

$$T_{\texttt{array\_last}}(n) = T_{\texttt{.elts}} + T_{\texttt{.len}} + T_{\texttt{-1}} + T_{\texttt{[]}} + T_{\texttt{return}}$$



Modeling `array_last`

```
def array_last(array, n):
    return array.elts[array.len - 1]
```

$T_{thing}(n)$ = time to do *thing* on an input of size n

T_{thing} = time to do *thing* that takes a fixed amount of time

$$T_{\text{array\_last}}(n) = T_{\texttt{.elts}} + T_{\texttt{.len}} + T_{\text{-1}} + T_{\texttt{[]}} + T_{\text{return}}$$
$$T_{\text{array\_last}}(n) = d_1$$

Let $d_1 = T_{\texttt{.elts}} + T_{\texttt{.len}} + T_{\text{-1}} + T_{\texttt{[]}} + T_{\text{return}}$

None of those depend on the size of the array. All are *constant*!

Modeling `array_last`

```
def array_last(array, n):
    return array.elts[array.len - 1]
```

$T_{thing}(n)$ = time to do *thing* on an input of size n

T_{thing} = time to do *thing* that takes a fixed amount of time

$$T_{\text{array\_last}}(n) = T_{\texttt{.elts}} + T_{\texttt{.len}} + T_{\text{-1}} + T_{\texttt{[]}} + T_{\text{return}}$$
$$T_{\text{array\_last}}(n) = d_1$$

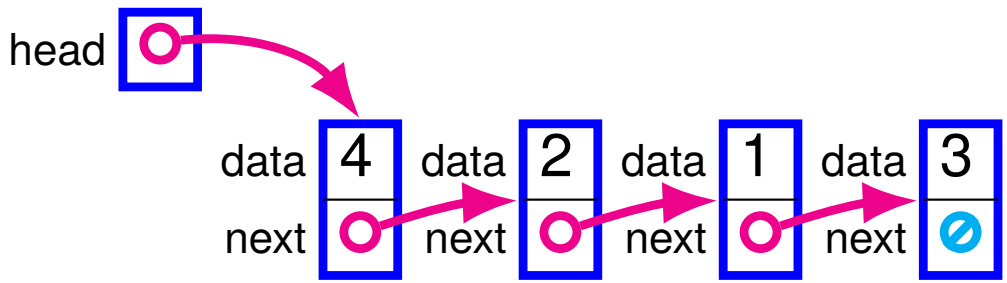
Let $d_1 = T_{\texttt{.elts}} + T_{\texttt{.len}} + T_{\text{-1}} + T_{\texttt{[]}} + T_{\text{return}}$

All work is done only once; does not depend on input!
$\rightarrow T_{\text{array\_last}}(n)$ is a *constant* function! Always the same value!

That's what we called **cheap** last time.

Modeling `list_last`

```
def list_last(lst):
    let cur = lst.head
    while cur.next is not None:
        cur = cur.next
    return cur.data
```



Modeling `list_last`

```
def list_last(lst):
    let cur = lst.head
    while cur.next is not None:
        cur = cur.next
    return cur.data
```

$$T_{\texttt{list\_last}}(n) =$$

Modeling `list_last`

```
def list_last(lst):
    let cur = lst.head
    while cur.next is not None:
        cur = cur.next
    return cur.data
```

$$T_{\texttt{list\_last}}(n) = T_{\texttt{.head}} +$$

Modeling `list_last`

```
def list_last(lst):
    let cur = lst.head
    while cur.next is not None:
        cur = cur.next
    return cur.data
```

$$T_{\texttt{list\_last}}(n) = T_{\texttt{.head}} + T_{\texttt{=}} +$$

Modeling `list_last`

```
def list_last(lst):
    let cur = lst.head
    while cur.next is not None:
        cur = cur.next
    return cur.data
```

$$T_{\texttt{list\_last}}(n) = T_{\texttt{.head}} + T_{\texttt{=}} + n \times T_{\texttt{.next}} +$$

Modeling `list_last`

```
def list_last(lst):
    let cur = lst.head
    while cur.next is not None:
        cur = cur.next
    return cur.data
```

$$T_{\texttt{list\_last}}(n) = T_{\texttt{.head}} + T_{\texttt{=}} + n \times T_{\texttt{.next}} + n \times T_{\texttt{is not None}}$$

Modeling `list_last`

```
def list_last(lst):
    let cur = lst.head
    while cur.next is not None:
        cur = cur.next
    return cur.data
```

$$T_{\texttt{list\_last}}(n) = T_{\texttt{.head}} + T_{\texttt{=}} + n \times T_{\texttt{.next}} + n \times T_{\texttt{is not None}} + n \times T_{\texttt{.next}} +$$

Modeling `list_last`

```
def list_last(lst):
    let cur = lst.head
    while cur.next is not None:
        cur = cur.next
    return cur.data
```

$$T_{\texttt{list\_last}}(n) = T_{\texttt{.head}} + T_{\texttt{=}} + n \times T_{\texttt{.next}} + n \times T_{\texttt{is not None}} + n \times T_{\texttt{.next}} + n \times T_{\texttt{=}} +$$

Modeling `list_last`

```
def list_last(lst):
    let cur = lst.head
    while cur.next is not None:
        cur = cur.next
    return cur.data
```

$$T_{\texttt{list\_last}}(n) = T_{\texttt{.head}} + T_{\texttt{=}} + n \times T_{\texttt{.next}} + n \times T_{\texttt{is not None}} + n \times T_{\texttt{.next}} + n \times T_{\texttt{=}} + T_{\texttt{.data}} +$$

Modeling `list_last`

```
def list_last(lst):
    let cur = lst.head
    while cur.next is not None:
        cur = cur.next
    return cur.data
```

$$T_{\texttt{list\_last}}(n) = T_{\texttt{.head}} + T_{\texttt{=}} + n \times T_{\texttt{.next}} + n \times T_{\texttt{is not None}} + n \times T_{\texttt{.next}} + n \times T_{\texttt{=}} + T_{\texttt{.data}} + T_{\texttt{return}}$$

Modeling `list_last`

```
def list_last(lst):
    let cur = lst.head
    while cur.next is not None:
        cur = cur.next
    return cur.data
```

$$T_{\texttt{list\_last}}(n) = T_{\texttt{.head}} + T_{\texttt{=}} + n \times T_{\texttt{.next}} + n \times T_{\texttt{is not None}} + n \times T_{\texttt{.next}} + n \times T_{\texttt{=}} + T_{\texttt{.data}} + T_{\texttt{return}}$$

Let $c_1 = T_{\texttt{.head}} + T_{\texttt{=}} + T_{\texttt{.data}} + T_{\texttt{return}}$

Modeling `list_last`

```
def list_last(lst):
    let cur = lst.head
    while cur.next is not None:
        cur = cur.next
    return cur.data
```

$$T_{\texttt{list\_last}}(n) = T_{\texttt{.head}} + T_{\texttt{=}} + n \times T_{\texttt{.next}} + n \times T_{\texttt{is not None}} + n \times T_{\texttt{.next}} + n \times T_{\texttt{=}} + T_{\texttt{.data}} + T_{\texttt{return}}$$

Let $c_1 = T_{\texttt{.head}} + T_{\texttt{=}} + T_{\texttt{.data}} + T_{\texttt{return}}$

Let $c_2 = T_{\texttt{.next}} + T_{\texttt{is not None}} + T_{\texttt{.next}} + T_{\texttt{=}}$

Modeling `list_last`

```
def list_last(lst):
    let cur = lst.head
    while cur.next is not None:
        cur = cur.next
    return cur.data
```

$$T_{\texttt{list\_last}}(n) = T_{\texttt{.head}} + T_{\texttt{=}} + n \times T_{\texttt{.next}} + n \times T_{\texttt{is not None}} + n \times T_{\texttt{.next}} + n \times T_{\texttt{=}} + T_{\texttt{.data}} + T_{\texttt{return}}$$

$$T_{\texttt{list\_last}}(n) = c_1 + c_2 n$$

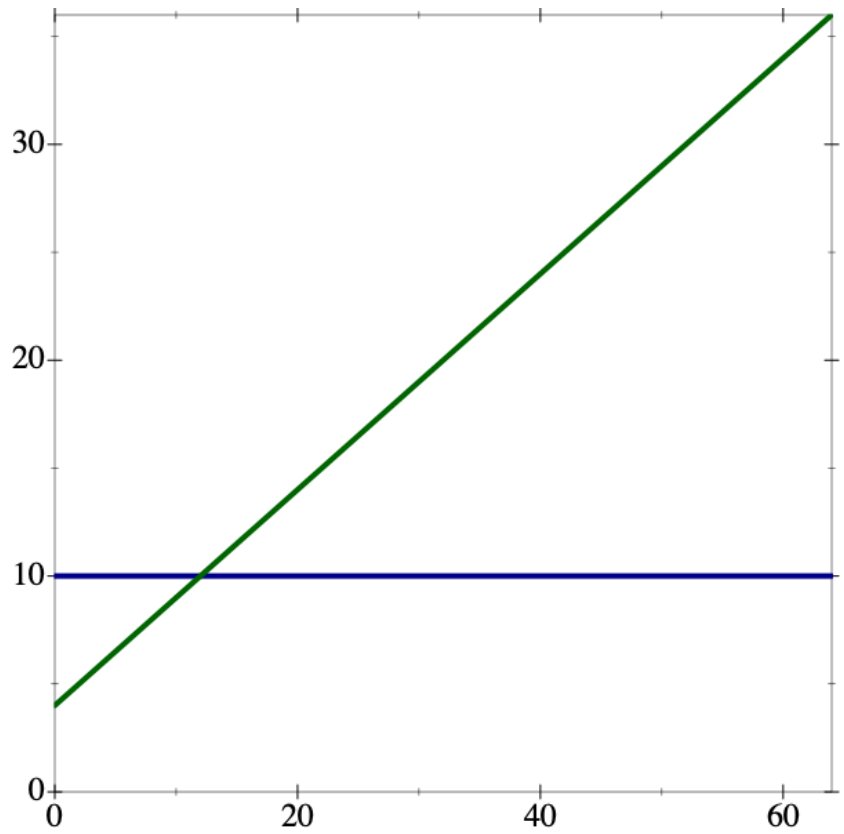
Let $c_1 = T_{\texttt{.head}} + T_{\texttt{=}} + T_{\texttt{.data}} + T_{\texttt{return}}$

Let $c_2 = T_{\texttt{.next}} + T_{\texttt{is not None}} + T_{\texttt{.next}} + T_{\texttt{=}}$

$T_{\texttt{list\_last}}(n)$ is proportional to $n \rightarrow$ *linear* function!

That's what we called **expensive** last time.

Comparing `array_last` and `list_last`

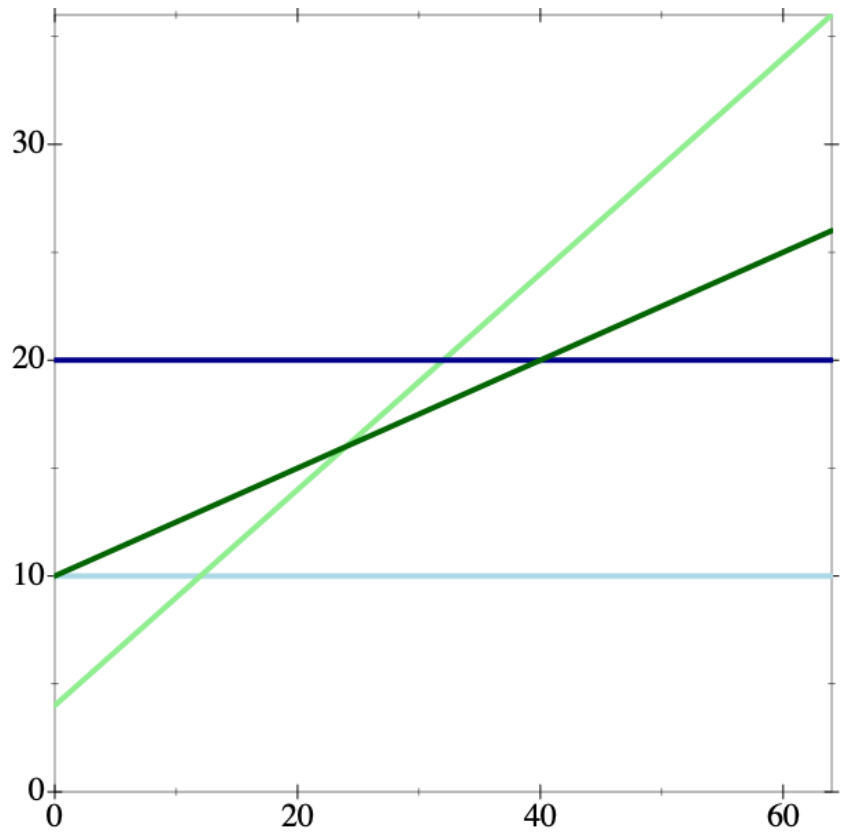


x axis n
y axis time

blue $T_{\texttt{array\_last}}(n) = 10$
green $T_{\texttt{list\_last}}(n) = 4 + 0.5\,n$

Let's pick concrete values for d_1, c_1, and c_2.
And let's try a few different combinations to see what changes (or doesn't).

Comparing `array_last` and `list_last`

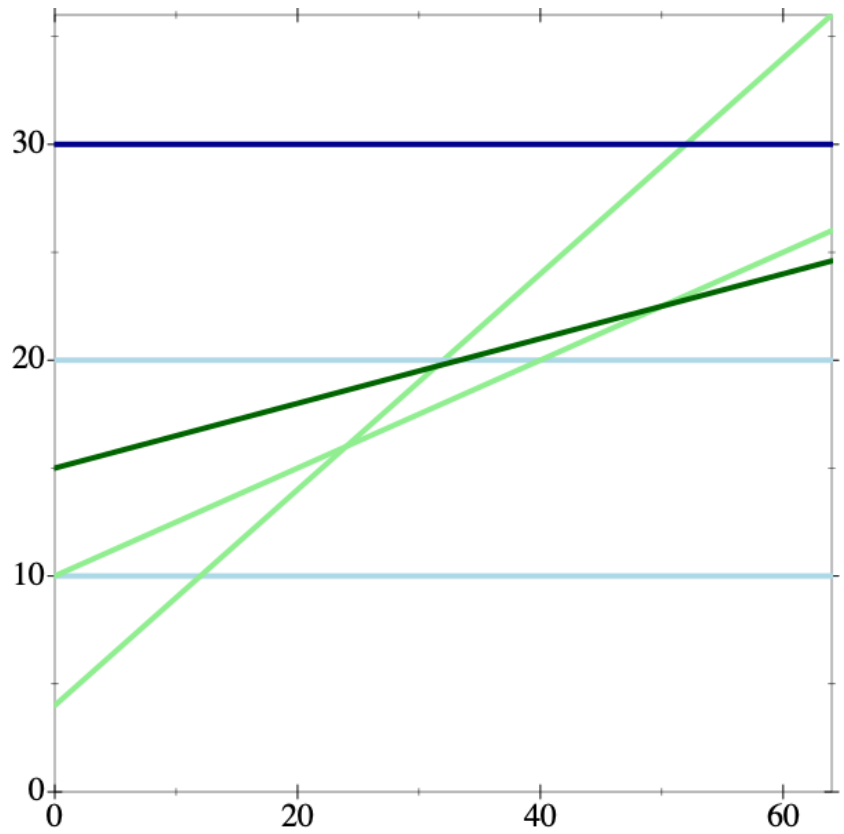


x axis n
y axis time

blue $T_{\texttt{array\_last}}(n) = 20$
green $T_{\texttt{list\_last}}(n) = 10 + 0.25\,n$

Even with different values, `array_last` never grows.
And `list_last` always has linear growth.

Comparing `array_last` and `list_last`

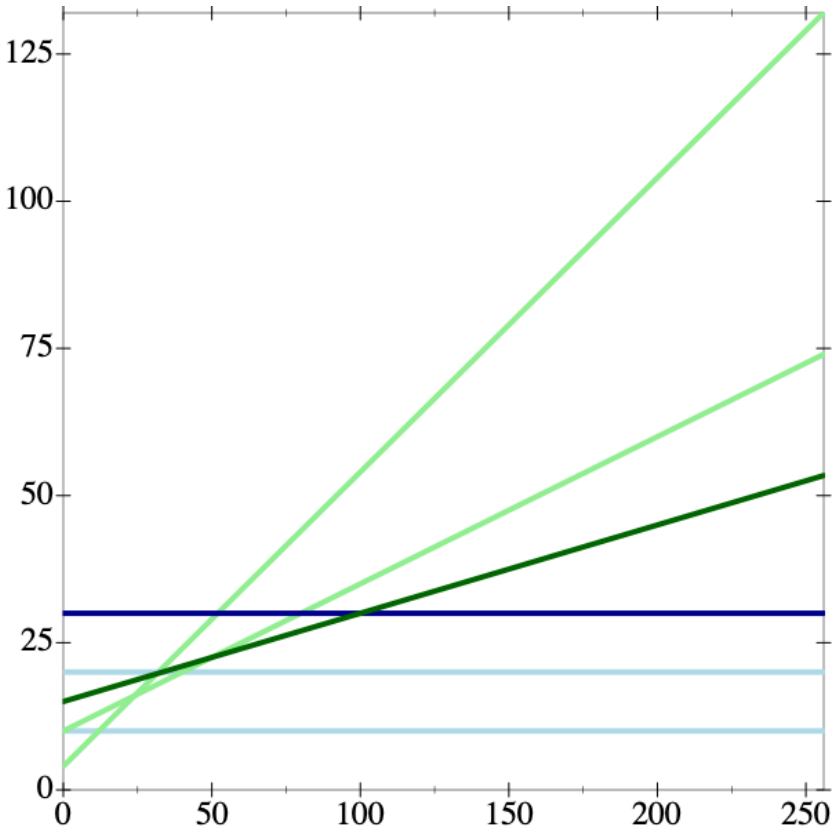


x axis n
y axis time

blue $T_{\texttt{array\_last}}(n) = 30$
green $T_{\texttt{list\_last}}(n) = 15 + 0.15\,n$

With a large enough d_1 and small enough c_1 and c_2, `array_last` may take more time than `list_last`.

Comparing `array_last` and `list_last`

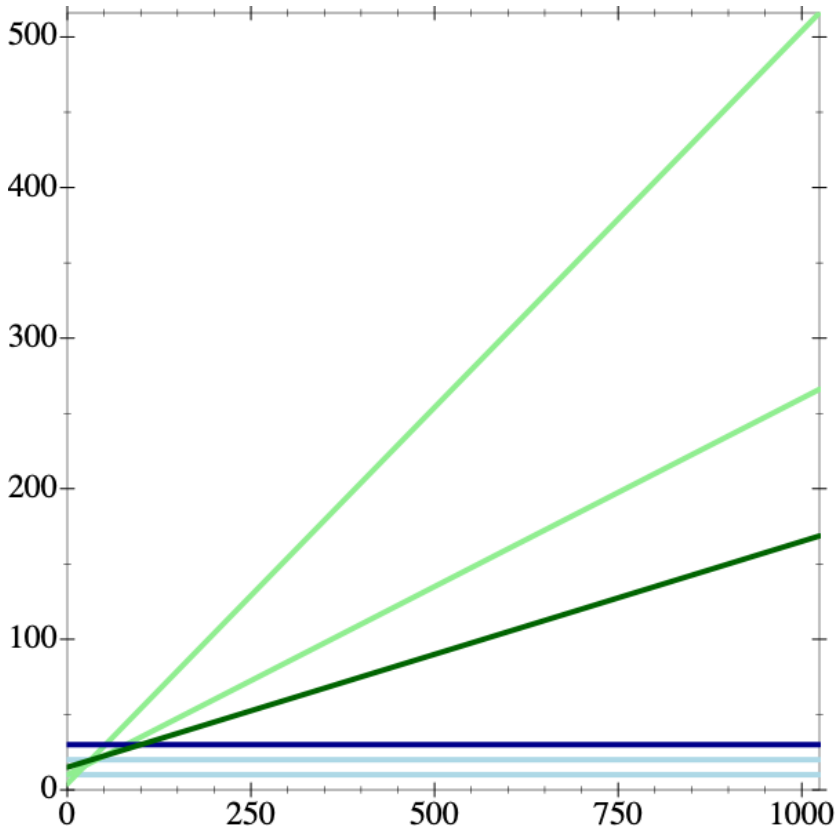


x axis n
y axis time

blue $T_{\texttt{array\_last}}(n) = 30$
green $T_{\texttt{list\_last}}(n) = 15 + 0.15\,n$

But when we zoom out and consider larger input sizes, `list_last` always ends up taking longer.

Comparing `array_last` and `list_last`



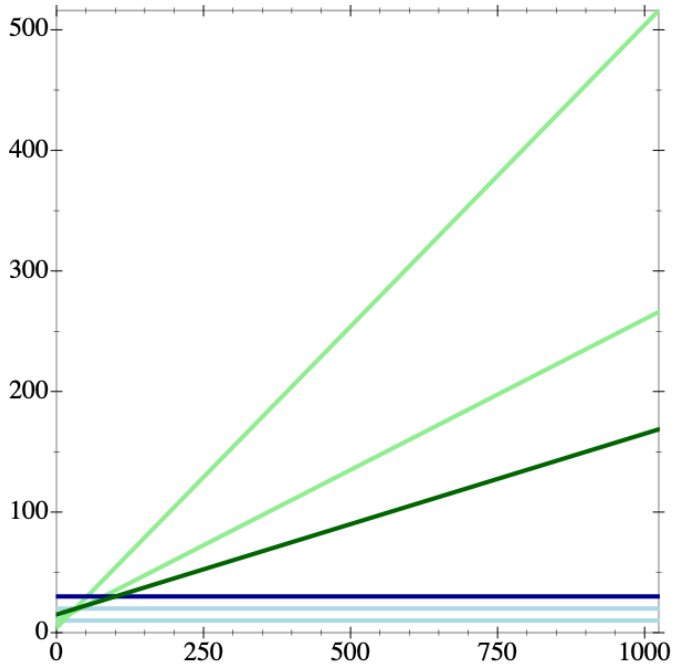
x axis n
y axis time

blue $T_{\texttt{array\_last}}(n) = 30$
green $T_{\texttt{list\_last}}(n) = 15 + 0.15\,n$

In general, we'll only care about large input sizes.
If your data structure is small, everything is fast anyway!

Large inputs = $n \to \infty$. Hence the name *asymptotic* complexity!

Big $\mathcal{O}$: Grouping Functions by Rates of Growth

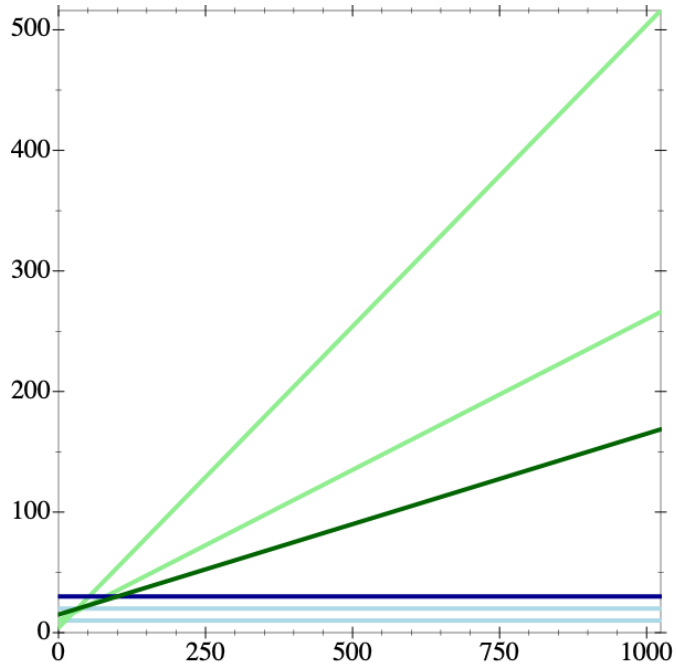


Even with different values for d_1...

$$f(n) = 10 \qquad f(n) = 20 \qquad f(n) = 30$$

...none of the blue lines grow at all.

Big $\mathcal{O}$: Grouping Functions by Rates of Growth

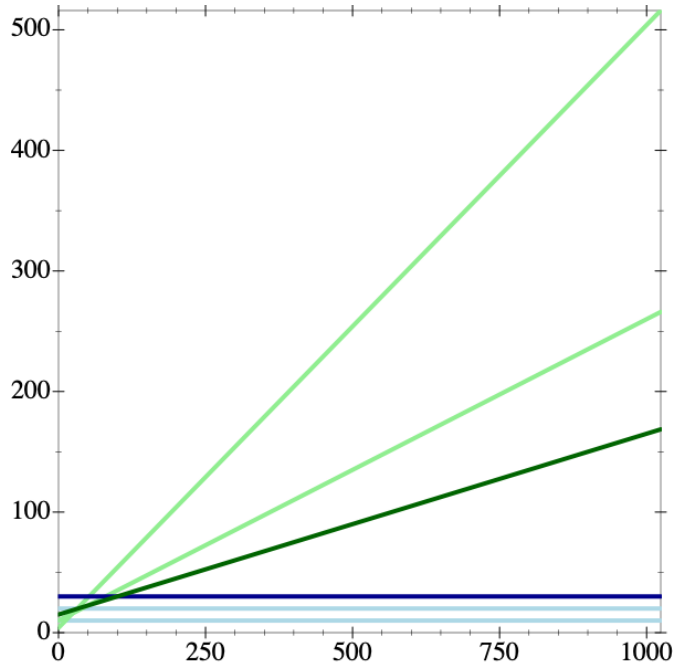


We will group all such functions into a big set: $\mathcal{O}(1)$
The set of all *constant* functions.

I.e., functions that grow in the same way $f(n) = 1$ does.
(i.e., they don't grow)

Operations whose time is described by a constant function (e.g., `array_last`) are said to take *constant time*.

Big $\mathcal{O}$: Grouping Functions by Rates of Growth



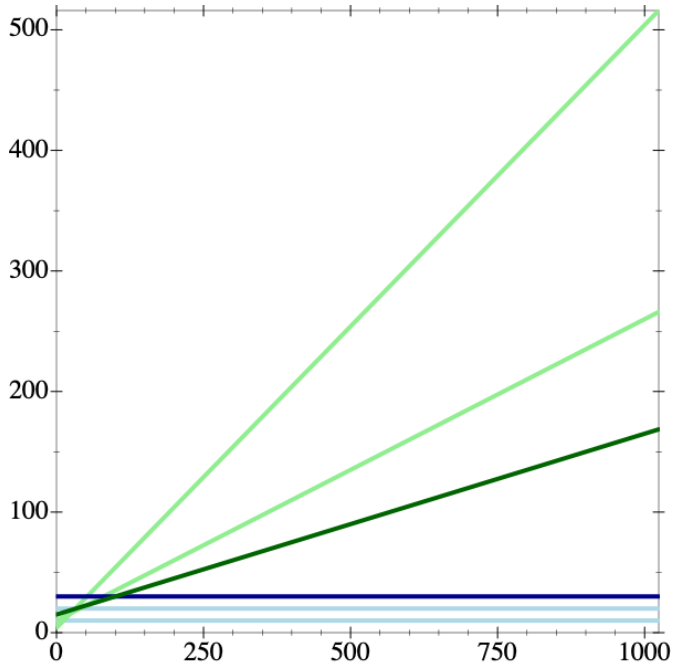
Even with different values for c_1 and c_2...

$f(n) = 4 + 0.5\,n$ $\quad$ $f(n) = 10 + 0.25\,n$ $\quad$ $f(n) = 15 + 0.15\,n$

...all the green lines double* when n doubles!

* except the c_1 part, which is too small to matter when n is big.

Big $\mathcal{O}$: Grouping Functions by Rates of Growth

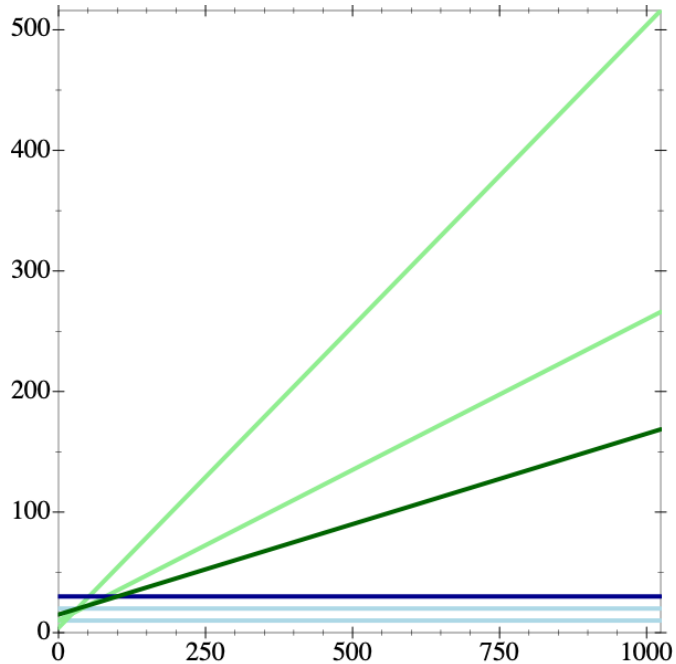


We will group all such functions into a big set: $\mathcal{O}(n)$
The set of all *linear* functions.

I.e., functions that grow in the same way $f(n) = n$ does.

Operations whose time is described by a linear function (e.g., `list_last`) are said to take *linear time*.

Big $\mathcal{O}$: Grouping Functions by Rates of Growth



For large inputs (i.e., the ones we care about),
operations that are $\mathcal{O}(1)$ are faster than ones that are $\mathcal{O}(n)$.

We will thus prefer the former over the latter whenever possible.

Formally

If f is a function, then $\mathcal{O}(f)$ is the set of functions that "grow no faster than" f

"g grows no faster than f" means there exist some constants c and m such that for all $n > m$, $\quad g(n) \leq c \times f(n)$

Let's unpack this:

- $\mathcal{O}(n)$ implicitly refers to the function $f(n) = n$, and so on.
- For all $n > m$ means: we only care about inputs larger than some constant m. I.e., large enough inputs, $n \to \infty$.
- Constant c means: we can change multiplicative factors, just like we changed c_2.

Stop! Question time!

Before we move on to another example...

Anything unclear so far?
Anything I missed?

Another Example: Searching a Sorted Array

We have a sorted array of n numbers.

We want to check if a given number (`target`) is in there.

```
def search (numbers, target):
    ...
```

I can think of two ways to do this. Can you?

Another Example: Searching a Sorted Array

We have a sorted array of n numbers.

We want to check if a given number (`target`) is in there.

```
def search (numbers, target):
    ...
```

I can think of two ways to do this. Can you?

- Look at each element one at a time (linear search)

Another Example: Searching a Sorted Array

We have a sorted array of n numbers.

We want to check if a given number (`target`) is in there.

```
def search (numbers, target):
    ...
```

I can think of two ways to do this. Can you?

- Look at each element one at a time (linear search)
- Oh, it's sorted. Binary search!

Linear Search

Look at each element in turn, check if it's the one we want.

```
def linear_search (numbers, target):
    for x in numbers:
        if x == target:
            return True
    return False
```

Linear Search

Look at each element in turn, check if it's the one we want.

```
def linear_search (numbers, target):
    for x in numbers:
        if x == target:
            return True
    return False
```

We don't know the position of our target, or if it's there at all!

But *in the worst case*, we need to look at all n numbers.

$$T_{\texttt{linear\_search}}(n) = T_{\text{setup } \texttt{for}} + n \times (T_{==} + T_{\text{next } \texttt{for}}) + T_{\text{return}}$$

Linear Search

Look at each element in turn, check if it's the one we want.

```
def linear_search (numbers, target):
    for x in numbers:
        if x == target:
            return True
    return False
```

We don't know the position of our target, or if it's there at all!

But *in the worst case*, we need to look at all n numbers.

$$T_{\texttt{linear\_search}}(n) = T_{\text{setup } \texttt{for}} + n \times (T_{==} + T_{\text{next } \texttt{for}}) + T_{\text{return}}$$
$$T_{\texttt{linear\_search}}(n) = c_1 + c_2 n$$

Linear Search

Look at each element in turn, check if it's the one we want.

```
def linear_search (numbers, target):
    for x in numbers:
        if x == target:
            return True
    return False
```

We don't know the position of our target, or if it's there at all!

But *in the worst case*, we need to look at all n numbers.

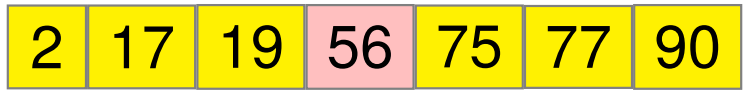
$$T_{\texttt{linear\_search}}(n) = T_{\text{setup } \texttt{for}} + n \times (T_{==} + T_{\text{next } \texttt{for}}) + T_{\text{return}}$$
$$T_{\texttt{linear\_search}}(n) = c_1 + c_2 n$$
$$T_{\texttt{linear\_search}}(n) = \mathcal{O}(n)$$

When analyzing the cost of operations, we're usually interested in the cost in the worst case. (I'll tell you when we're not.)

Binary Search

- Suppose we're looking up 75
- Initially, the element could be anywhere in the array

Binary Search



- Suppose we're looking up 75
- Initially, the element could be anywhere in the array
- Compare with the middle element

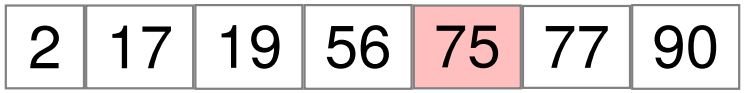
Binary Search

- Suppose we're looking up 75
- Initially, the element could be anywhere in the array
- Compare with the middle element
- Each iteration, possible range is cut in half

Binary Search

- Suppose we're looking up 75
- Initially, the element could be anywhere in the array
- Compare with the middle element
- Each iteration, possible range is cut in half

Binary Search



- Suppose we're looking up 75
- Initially, the element could be anywhere in the array
- Compare with the middle element
- Each iteration, possible range is cut in half
- When we're down to one element
  - Either we found what we're looking for
  - Or it's not in the array at all!

- `https://www.cs.usfca.edu/~galles/visualization/Search.html`

Binary Search

```
def binary_search (numbers, target):
    # look for `target` between indices `low` and `high`
    def helper (low, high):
        # empty range -> not found
        if low > high: return False
        let mid = (low + high) // 2
        if numbers[mid] == target: return True
        elif numbers[mid] < target:
            return helper(mid+1, high)
        else: # numbers[mid] > target:
            return helper(low, mid-1)
    return helper(0, numbers.len()-1)
```

We rely on sortedness to cut out half the numbers each step.

Key idea: like doing binary search on an array half the size!

Binary Search

```
def binary_search (numbers, target):
    # look for `target` between indices `low` and `high`
    def helper (low, high):
        # empty range -> not found
        if low > high: return False
        let mid = (low + high) // 2
        if numbers[mid] == target: return True
        elif numbers[mid] < target:
            return helper(mid+1, high)
        else: # numbers[mid] > target:
            return helper(low, mid-1)
    return helper(0, numbers.len()-1)
```

We only do *one* of the recursive calls.

$$T_{\texttt{binary\_search}}(n) = T_{\texttt{>}} + T_{\text{mid}} + T_{\texttt{[]}} + T_{\texttt{==}} + T_{\texttt{<}} + T_{\texttt{+/-}} + T_{\texttt{binary\_search}}(\tfrac{n}{2})$$

Binary Search

```
def binary_search (numbers, target):
    # look for `target` between indices `low` and `high`
    def helper (low, high):
        # empty range -> not found
        if low > high: return False
        let mid = (low + high) // 2
        if numbers[mid] == target: return True
        elif numbers[mid] < target:
            return helper(mid+1, high)
        else: # numbers[mid] > target:
            return helper(low, mid-1)
    return helper(0, numbers.len()-1)
```

We only do *one* of the recursive calls.

$$T_{\texttt{binary\_search}}(n) = T_{\texttt{>}} + T_{\texttt{mid}} + T_{\texttt{[]}} + T_{\texttt{==}} + T_{\texttt{<}} + T_{\texttt{+/-}} + T_{\texttt{binary\_search}}(\tfrac{n}{2})$$

$$T_{\texttt{binary\_search}}(n) = c_1 + T_{\texttt{binary\_search}}(\tfrac{n}{2})$$

Binary Search

```
def binary_search (numbers, target):
    # look for `target` between indices `low` and `high`
    def helper (low, high):
        # empty range -> not found
        if low > high: return False
        let mid = (low + high) // 2
        if numbers[mid] == target: return True
        elif numbers[mid] < target:
            return helper(mid+1, high)
        else: # numbers[mid] > target:
            return helper(low, mid-1)
    return helper(0, numbers.len()-1)
```

In the worst case, we get down to the empty range and don't find our target, and return `False`. That's our *base case*.

$$T_{\texttt{binary\_search}}(1) = T_{>}$$

Binary Search

```
def binary_search (numbers, target):
    # look for `target` between indices `low` and `high`
    def helper (low, high):
        # empty range -> not found
        if low > high: return False
        let mid = (low + high) // 2
        if numbers[mid] == target: return True
        elif numbers[mid] < target:
            return helper(mid+1, high)
        else: # numbers[mid] > target:
            return helper(low, mid-1)
    return helper(0, numbers.len()-1)
```

In the worst case, we get down to the empty range and don't find our target, and return `False`. That's our *base case*.

$$T_{\texttt{binary\_search}}(1) = T_{>}$$
$$T_{\texttt{binary\_search}}(1) = c_2$$

Binary Search

Putting all this together.

$$T_{\texttt{binary\_search}}(n) = c_1 + T_{\texttt{binary\_search}}(\tfrac{n}{2})$$

Binary Search

Putting all this together.

$$\begin{aligned} T_{\texttt{binary\_search}}(n) &= c_1 + T_{\texttt{binary\_search}}(\tfrac{n}{2}) \\ &= c_1 + c_1 + T_{\texttt{binary\_search}}(\tfrac{n}{4}) \end{aligned}$$

Binary Search

Putting all this together.

$$
\begin{aligned}
T_{\texttt{binary\_search}}(n) &= c_1 + T_{\texttt{binary\_search}}(\tfrac{n}{2}) \\
&= c_1 + c_1 + T_{\texttt{binary\_search}}(\tfrac{n}{4}) \\
&= c_1 + c_1 + c_1 + T_{\texttt{binary\_search}}(\tfrac{n}{8})
\end{aligned}
$$

Binary Search

Putting all this together.

$$
\begin{aligned}
T_{\texttt{binary\_search}}(n) &= c_1 + T_{\texttt{binary\_search}}(\tfrac{n}{2}) \\
&= c_1 + c_1 + T_{\texttt{binary\_search}}(\tfrac{n}{4}) \\
&= c_1 + c_1 + c_1 + T_{\texttt{binary\_search}}(\tfrac{n}{8}) \\
&\dots \\
&= c_1 + c_1 + c_1 + \dots + T_{\texttt{binary\_search}}(1)
\end{aligned}
$$

QUIZ: How many times do we divide by 2 before reaching 1?

A: 12 **B**: $log_2 n$ **C**: $\sqrt{n}$ **D**: n

Binary Search

Putting all this together.

$$
\begin{aligned}
T_{\texttt{binary\_search}}(n) &= c_1 + T_{\texttt{binary\_search}}(\tfrac{n}{2}) \\
&= c_1 + c_1 + T_{\texttt{binary\_search}}(\tfrac{n}{4}) \\
&= c_1 + c_1 + c_1 + T_{\texttt{binary\_search}}(\tfrac{n}{8}) \\
&\dots \\
&= c_1 + c_1 + c_1 + \dots + T_{\texttt{binary\_search}}(1) \\
&= c_1 \times \log_2 n + T_{\texttt{binary\_search}}(1)
\end{aligned}
$$

QUIZ: How many times do we divide by 2 before reaching 1?

A: 12 **B**: $log_2 n$ **C**: $\sqrt{n}$ **D**: n

Binary Search

Putting all this together.

$$
\begin{aligned}
T_{\texttt{binary\_search}}(n) &= c_1 + T_{\texttt{binary\_search}}(\tfrac{n}{2}) \\
&= c_1 + c_1 + T_{\texttt{binary\_search}}(\tfrac{n}{4}) \\
&= c_1 + c_1 + c_1 + T_{\texttt{binary\_search}}(\tfrac{n}{8}) \\
&\dots \\
&= c_1 + c_1 + c_1 + \dots + T_{\texttt{binary\_search}}(1) \\
&= c_1 \times \log_2 n + T_{\texttt{binary\_search}}(1) \\
&= c_1 \times \log_2 n + c_2
\end{aligned}
$$

QUIZ: How many times do we divide by 2 before reaching 1?

A: 12 **B**: $log_2 n$ **C**: $\sqrt{n}$ **D**: n

Binary Search

Putting all this together.

$$
\begin{aligned}
T_{\texttt{binary\_search}}(n) &= c_1 + T_{\texttt{binary\_search}}(\tfrac{n}{2}) \\
&= c_1 + c_1 + T_{\texttt{binary\_search}}(\tfrac{n}{4}) \\
&= c_1 + c_1 + c_1 + T_{\texttt{binary\_search}}(\tfrac{n}{8}) \\
&\dots \\
&= c_1 + c_1 + c_1 + \dots + T_{\texttt{binary\_search}}(1) \\
&= c_1 \times \log_2 n + T_{\texttt{binary\_search}}(1) \\
&= c_1 \times \log_2 n + c_2
\end{aligned}
$$

QUIZ: How many times do we divide by 2 before reaching 1?

A: 12 **B**: $log_2 n$ **C**: $\sqrt{n}$ **D**: n

So `binary_search` is in the set $\mathcal{O}(\log_2 n)$ (AKA *logarithmic*)

Linear Search versus Binary Search

Linear search takes $\mathcal{O}(n)$. Binary search takes $\mathcal{O}(\log_2 n)$.

What does this mean concretely?

n	$\log_2 n$
16	4
32	5
64	6
$65\,536$	16

Even as n gets larger, binary search remains pretty fast.

Linear Search versus Binary Search

Linear search takes $\mathcal{O}(n)$. Binary search takes $\mathcal{O}(\log_2 n)$.

What does this mean concretely?

n	$1\,000\,000 \times \log_2 n$
16	$4\,000\,000$
32	$5\,000\,000$
64	$6\,000\,000$
$65\,536$	$16\,000\,000$

Even as n gets larger, binary search remains pretty fast.

You can make the c_1 constant as large as you want...

Linear Search versus Binary Search

Linear search takes $\mathcal{O}(n)$. Binary search takes $\mathcal{O}(\log_2 n)$.

What does this mean concretely?

n	$1\,000\,000 \times \log_2 n$
16	$4\,000\,000$
32	$5\,000\,000$
64	$6\,000\,000$
$65\,536$	$16\,000\,000$
$1\,048\,576$	$20\,000\,000$

Even as n gets larger, binary search remains pretty fast.

You can make the c_1 constant as large as you want...

...but as n gets larger...

Linear Search versus Binary Search

Linear search takes $\mathcal{O}(n)$. Binary search takes $\mathcal{O}(\log_2 n)$.

What does this mean concretely?

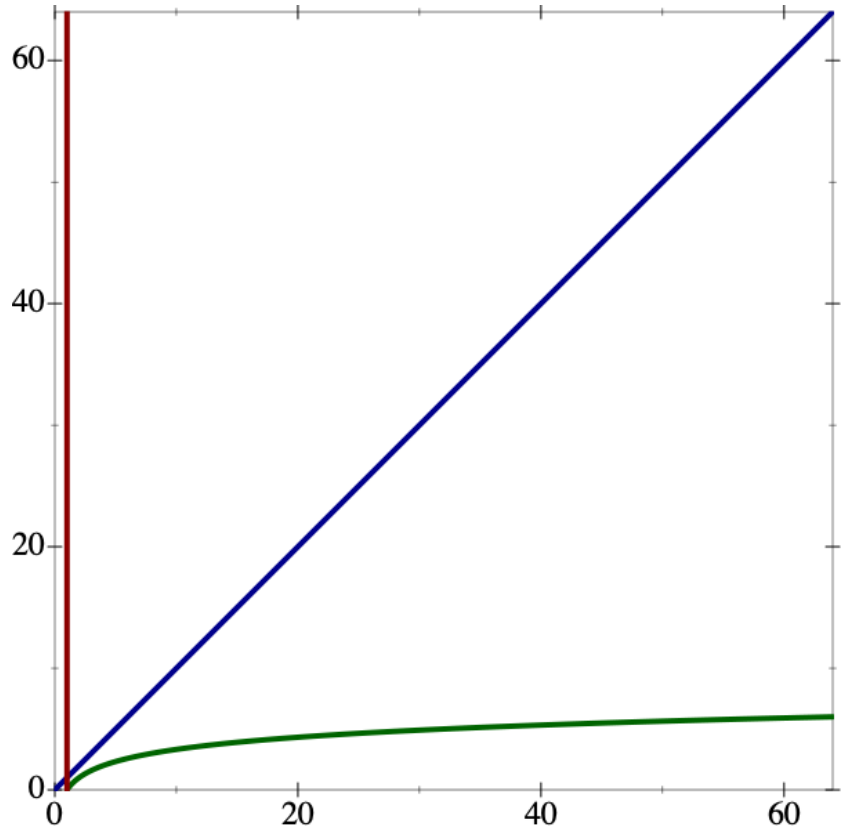
n	$1\,000\,000 \times \log_2 n$
16	$4\,000\,000$
32	$5\,000\,000$
64	$6\,000\,000$
$65\,536$	$16\,000\,000$
$1\,048\,576$	$20\,000\,000$
$33\,554\,432$	$25\,000\,000$
$4\,294\,967\,296$	$32\,000\,000$

Even as n gets larger, binary search remains pretty fast.

You can make the c_1 constant as large as you want...

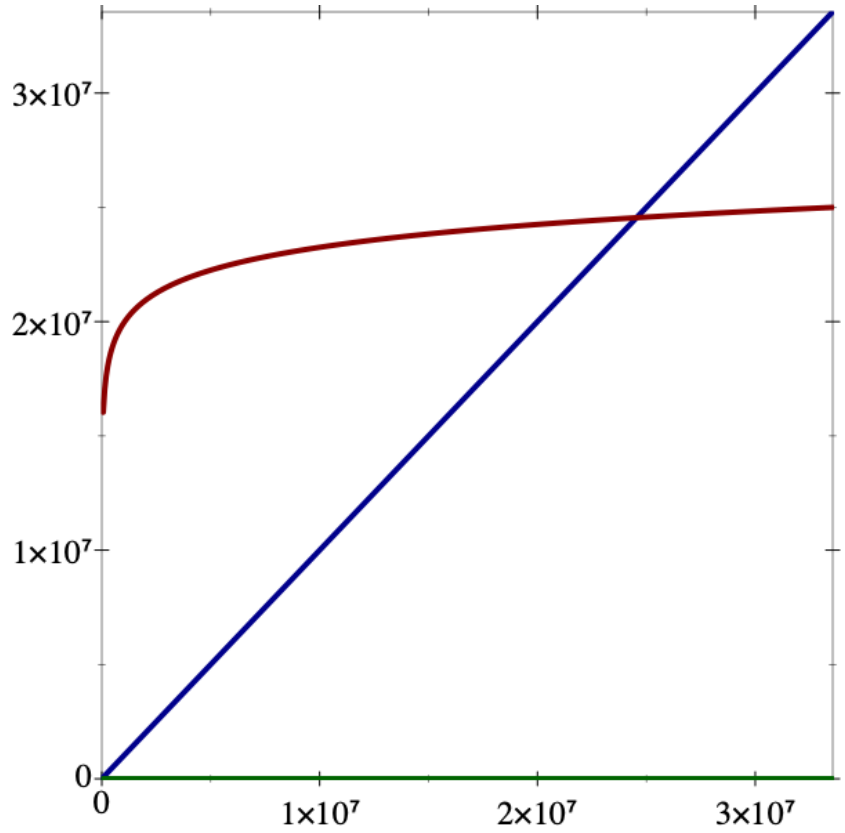
...but as n gets larger... binary search still wins eventually.

Linear Search versus Binary Search



red	$1\,000\,000 \times \log_2 n$
blue	n
green	$\log_2 n$

Linear Search versus Binary Search



red	$1\,000\,000 \times \log_2 n$
blue	n
green	$\log_2 n$

Another Definition

$f \lll g$ means $f \in \mathcal{O}(g)$ but $g \notin \mathcal{O}(f)$

For example:

$$\log_2 n \lll n$$

because:

$$\log_2 n \in \mathcal{O}(n)$$

$$n \notin \mathcal{O}(\log_2 n)$$

Another Definition

$f \lll g$ means $f \in \mathcal{O}(g)$ but $g \notin \mathcal{O}(f)$

For example:

$$\log_2 n \lll n$$

because:

$$\log_2 n \in \mathcal{O}(n)$$

$$n \notin \mathcal{O}(\log_2 n)$$

Also:

$$1 \lll \log_2 n$$

Big-O Equalities

Here are some rules we can apply to simplify complexity expressions (some of which we used already):

- $\mathcal{O}(f(n) + c) = \mathcal{O}(f(n))$ E.g., $\mathcal{O}(n + 4) = \mathcal{O}(n)$
  Equivalently: $f(n) + c \in \mathcal{O}(f(n))$
  In other words: adding constants doesn't matter.

Big-O Equalities

Here are some rules we can apply to simplify complexity expressions (some of which we used already):

- $\mathcal{O}(f(n) + c) = \mathcal{O}(f(n))$ E.g., $\mathcal{O}(n + 4) = \mathcal{O}(n)$
  Equivalently: $f(n) + c \in \mathcal{O}(f(n))$
  In other words: adding constants doesn't matter.

- $\mathcal{O}(c \times f(n)) = \mathcal{O}(f(n))$ E.g., $\mathcal{O}(4n) = \mathcal{O}(n)$
  In other words: multiplying by constants doesn't matter.

Big-O Equalities

Here are some rules we can apply to simplify complexity expressions (some of which we used already):

- $\mathcal{O}(f(n) + c) = \mathcal{O}(f(n))$ E.g., $\mathcal{O}(n + 4) = \mathcal{O}(n)$
  Equivalently: $f(n) + c \in \mathcal{O}(f(n))$
  In other words: adding constants doesn't matter.

- $\mathcal{O}(c \times f(n)) = \mathcal{O}(f(n))$ E.g., $\mathcal{O}(4n) = \mathcal{O}(n)$
  In other words: multiplying by constants doesn't matter.

- $\mathcal{O}(\log_k f(n)) = \mathcal{O}(\log_j f(n))$ E.g., $\mathcal{O}(\log_2 n) = \mathcal{O}(\log_3 n)$
  In other words: bases don't matter. (base change = $\times c$)
  (So we'll stop bothering with them.)

Big-O Equalities

Here are some rules we can apply to simplify complexity expressions (some of which we used already):

- $\mathcal{O}(f(n) + c) = \mathcal{O}(f(n))$ E.g., $\mathcal{O}(n + 4) = \mathcal{O}(n)$
  Equivalently: $f(n) + c \in \mathcal{O}(f(n))$
  In other words: adding constants doesn't matter.

- $\mathcal{O}(c \times f(n)) = \mathcal{O}(f(n))$ E.g., $\mathcal{O}(4n) = \mathcal{O}(n)$
  In other words: multiplying by constants doesn't matter.

- $\mathcal{O}(\log_k f(n)) = \mathcal{O}(\log_j f(n))$ E.g., $\mathcal{O}(\log_2 n) = \mathcal{O}(\log_3 n)$
  In other words: bases don't matter. (base change = $\times c$)
  (So we'll stop bothering with them.)

- $\mathcal{O}(f(n) + g(n)) = \mathcal{O}(f(n))$ if $g \lll f$
  E.g., $\mathcal{O}(n + \log n) = \mathcal{O}(n)$
  In other words: adding smaller things doesn't matter.
  (Implies the first rule.)

Big-O Equalities

Always use the most specific complexity expression you can.

For example:

$\mathcal{O}(3n + 4\log_2 n + 5) = \mathcal{O}(n)$

- So just say $\mathcal{O}(n)$

$\mathcal{O}(1) \subset \mathcal{O}(n)$

- Correct, constant funcs grow no faster than linear funcs
- But $\mathcal{O}(1)$ is more specific than $\mathcal{O}(n)$, so use that
- Most things we care about are technically $\mathcal{O}(2^n)$
  - But that's a pretty useless thing to say…
  - And makes things sound worse than they are!
  - So be as precise as you can!

Big-O Inequalities

For constants j, k where $j < k$:

$1 \lll \log n$ constants are less than logs…

Big-O Inequalities

For constants j, k where $j < k$:

$$1 \lll \log n \qquad \text{constants are less than logs…}$$

$$\lll n^j \qquad \text{are less than polynomials…}$$

Big-O Inequalities

For constants j, k where $j < k$:

$$1 \lll \log n \qquad \text{constants are less than logs…}$$
$$\lll n^j \qquad \text{are less than polynomials…}$$
$$\lll n^k \qquad \text{are less than higher-degree polynomials…}$$

Big-O Inequalities

For constants j, k where $j < k$:

$$1 \lll \log n \qquad \text{constants are less than logs…}$$
$$\lll n^j \qquad \text{are less than polynomials…}$$
$$\lll n^k \qquad \text{are less than higher-degree polynomials…}$$
$$\lll n^k \log n \qquad \text{are less than poly-log (for same } k\text{)…}$$

Big-O Inequalities

For constants j, k where $j < k$:

$$1 \lll \log n \qquad \text{constants are less than logs…}$$
$$\lll n^j \qquad \text{are less than polynomials…}$$
$$\lll n^k \qquad \text{are less than higher-degree polynomials…}$$
$$\lll n^k \log n \qquad \text{are less than poly-log (for same } k\text{)…}$$
$$\lll j^n \qquad \text{are less than exponentials…}$$

Big-O Inequalities

For constants j, k where $j < k$:

$$
\begin{aligned}
1 &\lll \log n && \text{constants are less than logs…} \\
&\lll n^j && \text{are less than polynomials…} \\
&\lll n^k && \text{are less than higher-degree polynomials…} \\
&\lll n^k \log n && \text{are less than poly-log (for same } k\text{)…} \\
&\lll j^n && \text{are less than exponentials…} \\
&\lll k^n && \text{are less than higher-base exponentials…}
\end{aligned}
$$

Big-O Inequalities

For constants j, k where $j < k$:

$$
\begin{aligned}
1 &\lll \log n && \text{constants are less than logs…} \\
&\lll n^j && \text{are less than polynomials…} \\
&\lll n^k && \text{are less than higher-degree polynomials…} \\
&\lll n^k \log n && \text{are less than poly-log (for same } k\text{)…} \\
&\lll j^n && \text{are less than exponentials…} \\
&\lll k^n && \text{are less than higher-base exponentials…} \\
&\lll n^n && \text{are less than this horror.}
\end{aligned}
$$

Big-O Inequalities

For constants j, k where $j < k$:

$$
\begin{aligned}
1 &\lll \log n && \text{constants are less than logs…} \\
&\lll n^j && \text{are less than polynomials…} \\
&\lll n^k && \text{are less than higher-degree polynomials…} \\
&\lll n^k \log n && \text{are less than poly-log (for same } k\text{)…} \\
&\lll j^n && \text{are less than exponentials…} \\
&\lll k^n && \text{are less than higher-base exponentials…} \\
&\lll n^n && \text{are less than this horror.}
\end{aligned}
$$

In practice, worse than n^3 is intractable. j^n is a non-starter.

Above are the most common classes; we will see others.

For More Examples

- Check out the complexity supplementary video on Canvas
- We will analyze sorting functions next time
- Do the worksheet!

Next time: Sorting